Backend
Delivery Pizzaria

ILES/ULBRA - Itumbiara

Introdução

- vamos desenvolver um backend para gerenciar um sistema de delivery de pizzaria.
- O foco será criar rotas para gerenciar pedidos, como **criar um pedido**, **adicionar ou remover produtos**, **escolher opções**, **finalizar e entregar pedidos**, além de **autenticar usuários**.
- Essas funcionalidades serão definidas em arquivos de rotas, que dependem de estruturas como **banco de dados**, **modelos**, **dependências** e **esquemas** para funcionar eficientemente.

Estruturas do Projeto

- **Integração com Banco de Dados no FastAPI**: Um banco de dados será criado para armazenar pedidos, usuários e itens de pedidos, garantindo que todas as informações sejam persistentes e acessíveis.
- **Modelos de Dados (Models) no FastAPI**: Arquivos de modelos definirão as tabelas do banco de dados, como tabelas para usuários, pedidos e itens de pedidos, estruturando os dados de forma organizada.
- **Arquivo de Dependências: Regras de Acesso e Autenticação**: Este arquivo permitirá restringir o acesso e gerenciar requisições ao banco de dados de forma eficiente, em cenários de múltiplas requisições simultâneas.
- **Schemas no FastAPI: Estruturando Dados de Entrada e Resposta**: Os esquemas estruturam os dados de entrada e saída, garantindo consistência e eficiência no processamento.
- **Tipagem de Dados no FastAPI: Segurança e Performance**: A tipagem estática do FastAPI melhora a performance e a segurança, evitando erros de tipo durante a execução.

FastAPI

- Uma das grandes vantagens do FastAPI é o uso de funções assíncronas, que permitem processar múltiplas requisições simultaneamente. Imagine que uma requisição de um usuário leve 15 segundos para consultar o banco de dados.
- Em vez de travar o sistema esperando essa resposta, o FastAPI inicia a próxima requisição e retorna à anterior quando possível. Isso evita gargalos e melhora a escalabilidade, sendo ideal para sistemas com alto tráfego, como uma pizzaria delivery.

https://fastapi.tiangolo.com/

Criando o Ambiente Virtual - .venv

- Sem isolamento, todas as bibliotecas são instaladas na instalação global do Python.
- Isso gera três problemas previsíveis:
  - **Conflito de versões:** o Projeto A precisa de pacotes 1.5, o Projeto B de pacotes 2.2. Sem isolamento, um dos dois quebra.
  - **Irreprodutibilidade:** o código roda na sua máquina e falha na do colega, porque ninguém sabe exatamente o que está instalado.
  - **Poluição do sistema:** desinstalar dependências vira tentativa e erro.
- **O ambiente virtual (venv)** cria uma pasta local com um interpretador Python próprio e um diretório site-packages isolado.

Preparação do Ambiente

- Instalar do Python
- Instalar o VS Code e as extensões:
- **Python:**Detecção de interpretadores, execução, depuração
- **Pylance:** Microsoft; IntelliSense e checagem de tipos (instalada junto com a anterior)

PowerShell – Liberar os Scripts

- Este é **o erro nº 1** de quem usa Windows. O PowerShell bloqueia scripts por padrão, e o comando de ativação do venv é um script .ps1.
- Abra o PowerShell (não precisa ser como administrador) e execute **uma única vez**:
- *Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned*
- Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned

Criando o Ambiente Virtual

- Abra a **pasta do projeto** no VS Code (Arquivo → Abrir Pasta). Nunca abra um arquivo .py solto.
- Ctrl + Shift + P → digite **Python: Create Environment**
- Selecione **Venv**
- Escolha a versão do Python (ex.: Python 3.12.x)

Selecionar o Interpretador no VS code - B

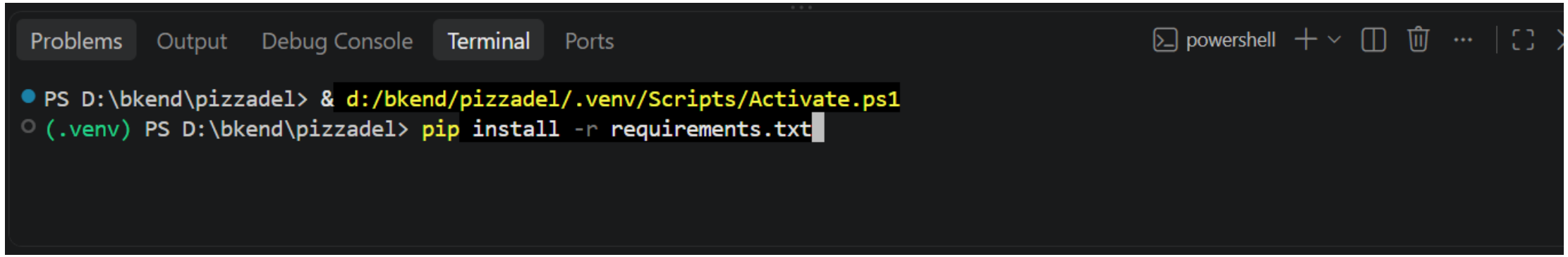
- # 1. Navegue até a pasta do projeto
  - cd C:\Projetos\meu-projeto
- # 2. Crie o ambiente
  - py -m venv .venv
- # 3. Ative o ambiente
  - .\.venv\Scripts\Activate.ps1

Selecionar o Interpretador no VS code

- Ctrl + Shift + P → **Python: Select Interpreter**
- Escolha a entrada marcada com **('.venv': venv)** e caminho .\.venv\Scripts\python.exe
- **Validação visual:** a barra de status inferior direita deve exibir algo como 3.12.4 ('.venv': venv).
- **Validação por terminal** (o teste definitivo):
- python -c "import sys; print(sys.executable)“

Iniciando o Projeto do Zero

- Abra o VS Code com um ambiente virtual configurado. Crie um arquivo chamado main.py, que será o ponto de entrada do projeto. A primeira etapa é instalar as bibliotecas necessárias:



Executando o Servidor FastAPI

main.py

```python
# uvicorn main:app --reload - Rodar o Código

from fastapi import FastAPI

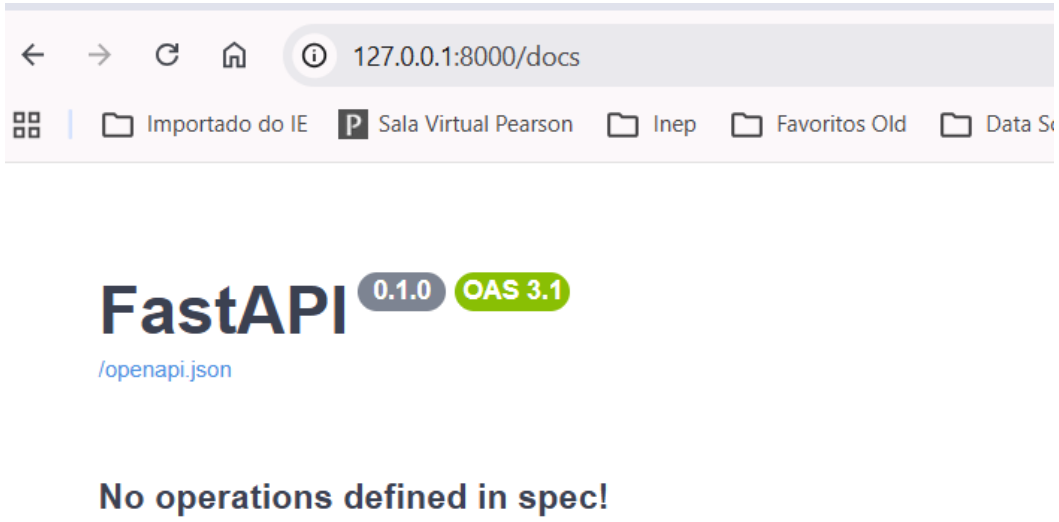
app = FastAPI()

```

```
uvicorn main:app --reload
```

Executando o Servidor FastAPI

- Ao executar o comando uvicorn main:app –reload, o servidor será iniciado localmente (geralmente em http://127.0.0.1:8000). Acesse esse link no navegador para verificar que o servidor está ativo.



Criando e Testando Rotas (Pedidos e Autenticação)

- Rotas, ou endpoints, definem o que acontece quando um usuário acessa um link específico da API, como *dominio.com/pedidos*.
- No FastAPI, rotas são os caminhos que vêm após o domínio do site, e cada rota está associada a um tipo de requisição HTTP.
- Por exemplo, ao acessar localhost:8000/pedidos, a API deve responder com uma ação predefinida, como listar pedidos.
- Como estamos construindo uma API REST, essas rotas lidam com requisições e respostas no formato JSON, sem a camada de frontend.

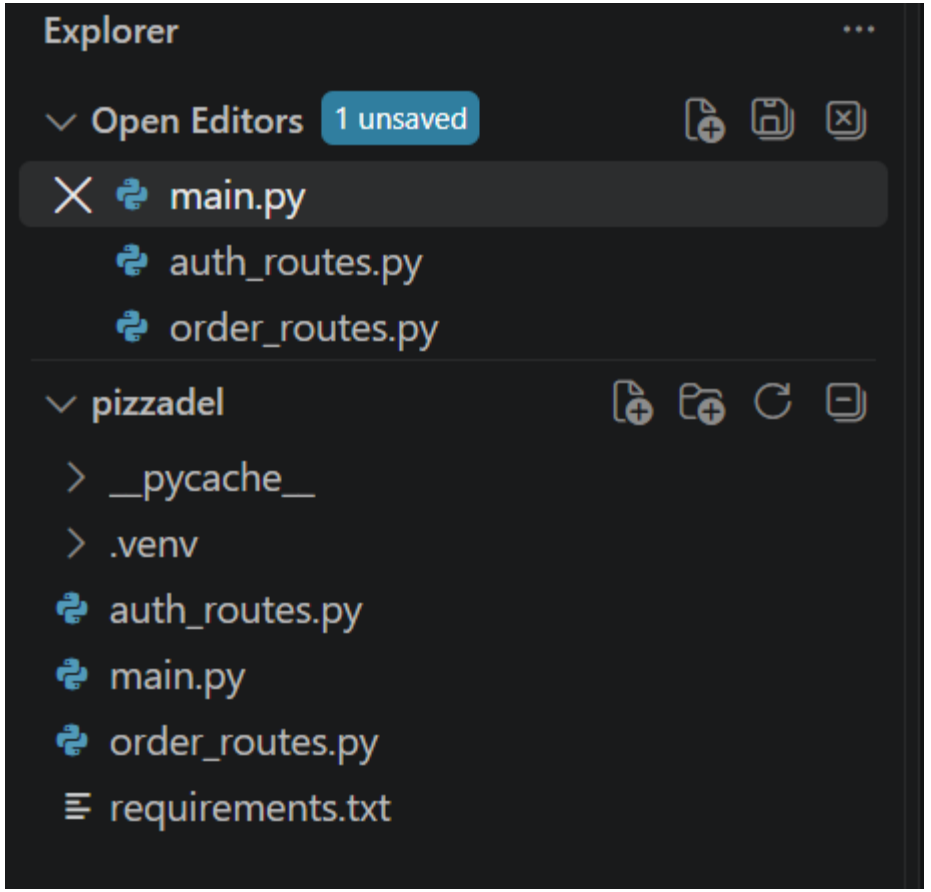
Entendendo as Requisições

- APIs REST geralmente seguem o padrão CRUD, que se traduz em quatro tipos principais de requisições HTTP:
- **GET**: Lê informações (ex.: listar pedidos).
- **POST**: Cria ou envia dados (ex.: criar um pedido).
- **PUT/PATCH**: Edita dados existentes (ex.: atualizar ou modificar um pedido).
- **DELETE**: Remove dados (ex.: excluir um pedido).
- Neste projeto vamos focar em requisições GET e POST, as mais comuns, mas você também aprenderá como implementar PUT e DELETE se necessário.
- Por exemplo, uma rota GET em /pedidos pode listar todos os pedidos, enquanto uma POST em /pedidos/criar pode adicionar um novo pedido.

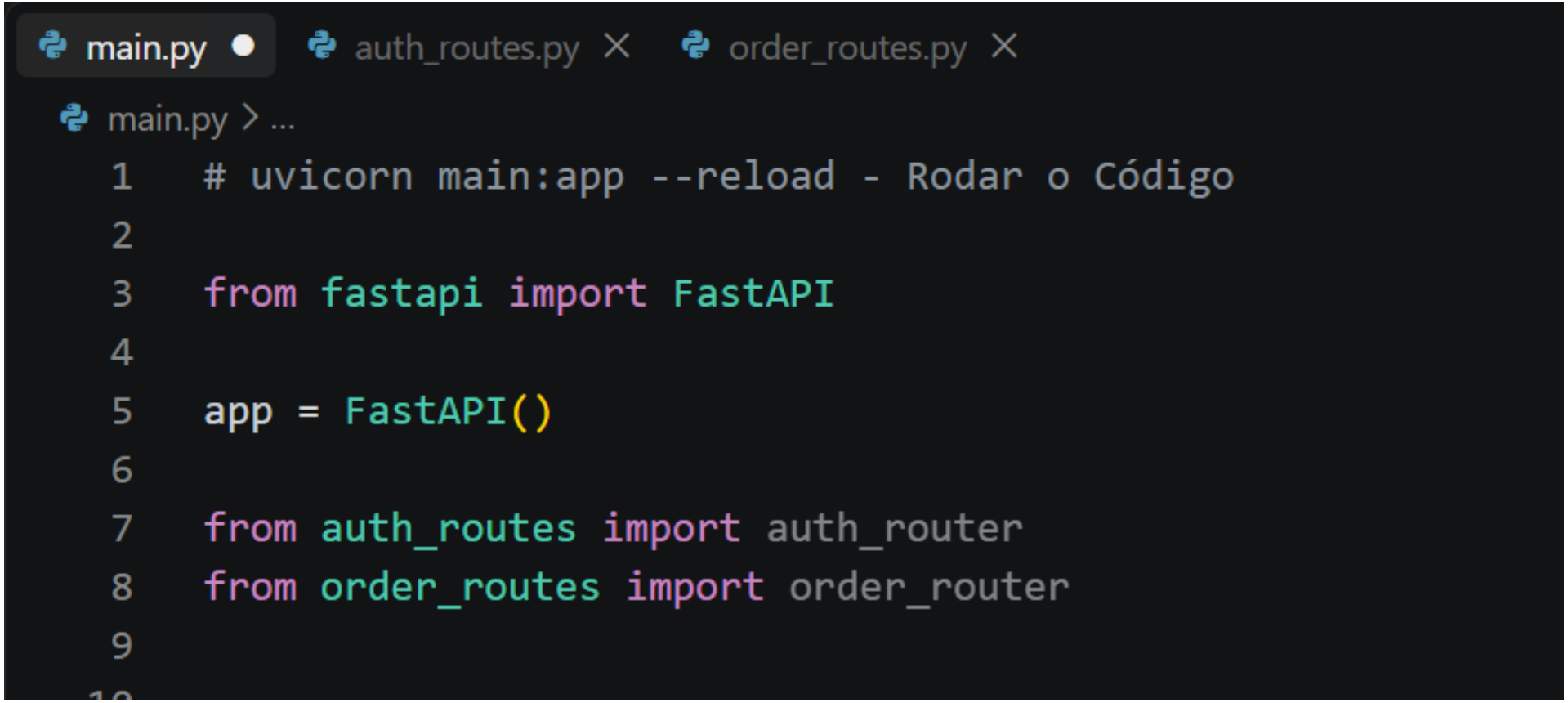
Criando as Rotas

- Para organizar o projeto, vamos imaginar arquivos de rotas separados: auth_routes.py (para autenticação) e order_routes.py (para pedidos).
- Essa separação evita conflitos e melhora a organização, especialmente em projetos com diferentes seções, como autenticação, pedidos e, futuramente, uma área administrativa.
- Cada arquivo terá um prefixo único para suas rotas (ex.: /auth e /pedidos), garantindo que não haja sobreposição de caminhos.

Criando os Arquivos – Autenticação e Rotas



Importando Rotas para o arquivo main.py



Criando os Roteadores

main.py ● auth_routes.py ● order_routes.py ●

auth_routes.py > ...

```python
from fastapi import APIRouter

auth_router = APIRouter(prefix="/auth", tags=["auth"])

```

main.py ● auth_routes.py ● order_routes.py ●

order_routes.py > ...

```python
from fastapi import APIRouter

order_router = APIRouter(prefix="/pedidos", tags=["pedidos"])

```

Incluindo as Rotas no site

```
#Incluindo as Rotas no main.py


app.include_router(auth_router)
app.include_router(order_router)
```

Criando a Estrutura de Rotas

```python
# Criando a Rota de Pedidos (order_routes)
@order_router.get("/lista") #@ Decorator
async def pedidos():
    return {"Mensagem":"Você Acessou a Rota de Pedidos"}
```

```python
# Criando a Rota de Autenticação - auth_routes.py

@auth_router.get("/")
async def autenticar():
    return {"Mensagem" : "Acessou a Autenticação", "Autenticado" : False}

```

Ajustes de Mensagem - WordStrings

```python
# Criando a Rota de Pedidos (order_routes)
@order_router.get("/lista") #@ Decorator
async def pedidos():
    """
       Acesso a rota de Pedidos
    """
    return {"Mensagem":"Você Acessou a Rota de Pedidos"}
```

```python
# Criando a Rota de Autenticação - auth_routes.py

@auth_router.get("/")
async def autenticar():
    """
       Acesso a rota de Autenticação
    """
    return {"Mensagem" : "Acessou a Autenticação", "Autenticado" : False}
```